LANGUAGES AND AUTOMATA I

Final Exam Answers

1. In what sense are Python anonymous functions "limited" compared to those of JavaScript?

   The body of a Python lambda can only be a single expression.

2. How do Go and Rust differ in their implementation of *errors-as-values*? That is, what is each language's preferred way to write and invoke functions can either succeed or fail? Explain precisely. You don't *have* to give code examples, but they may help compensate somewhat for inaccurate wording.

   Go uses **multiple return values**.
   Rust uses a **single value** of the sum type `Result`.

3. It is tempting to return a `Result` object from a find/search/lookup operation, but doing so improperly expresses what the operation is really all about. What should you do instead and why?

   You should throw an exception or **return an option value**, because we want to know if something is *present* or not. Not being present is *not an error*.

4. C, C++, Java, JavaScript, Kotlin, and many other languages have no need for a walrus operator, yet Python has one. Why was such an operator added to Python? That is, in what sense (exactly) is Python trying to differentiate itself from those other languages?

   In C, C++, Java, JavaScript, Kotlin, etc., the assignment operator (`=`) can appear in both expressions and statements. Python wanted to **visibly differentiate** expression-based-assignment `:=` from statement-based-assignment (`=`) by having two different operators.

5. Give a type theoretic definition, using the notation from class, for a game show prize that can be either a rock, cash (in some dollar amount), a car (with a make, model, and year), or a lottery

ticket (with a list of numbers).

$$\frac{}{\textsf{rock}: \textit{Prize}} \qquad \frac{n\textsf{: Nat}}{\textsf{cash } n\textsf{: Prize}} \qquad \frac{\textit{make}\textsf{: String} \quad \textit{model}\textsf{: String} \quad \textit{year}\textsf{: Nat}}{\textsf{car } \textit{make}\ \textit{model}\ \textit{year}\textsf{: Prize}} \qquad \frac{\textit{nums}\textsf{: Nat}^*}{\textsf{ticket } \textit{nums}\textsf{: Prize}}$$

6. Given types T and U where $x\textsf{: } T$, $y\textsf{: } T$, $a\textsf{: } U$, $b\textsf{: } U$, $c\textsf{: } U$, give (a) the complete set of inhabitants of the type $T + U$, and (b) the complete set of inhabitants of $T \times U$.

   (a) $T + U$ has $\textsf{tag}_1 x$, $\textsf{tag}_1 y$, $\textsf{tag}_2 a$, $\textsf{tag}_2 b$, and $\textsf{tag}_2 c$.
   (b) $T \times U$ has (x, a), (x, b), (x, c), (y, a), (y, b), and (y, c).

7. Give three features of C that earn it the label **memory unsafe**.

   C has **pointer arithmetic**, **no bounds checking**, and **manual memory deallocation**.

8. How do Go, Swift, and Rust, respectively, achieve memory safety? (You can just name their strategies rather than explaining the details of each.)

   Go uses **garbage collection**, Swift uses **automatic reference counting**, and Rust uses **ownership and borrowing**.

9. Believe it or not, Java doesn't have default parameters. What are you supposed to use to get the same effect?

   Method overloading.

10. JavaScript uses the `...` symbol for two completely different purposes. What are these two purposes, exactly? Give the official name of each and a **one-line** example usage for each.

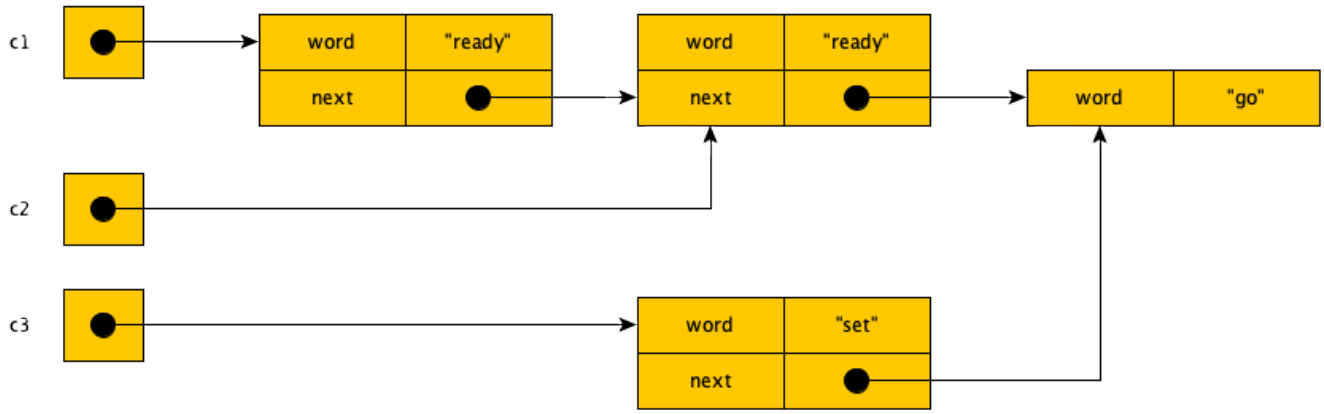
    The **spread operator** is used to *expand* an array into a list of arguments, e.g., `Math.max(...a)`, where `a` is an array. The **rest parameter** is used to *collect* arguments into an array, e.g., `function f(...args) {return args}`.

11. Can the C++ function `int f() {int x=2; int& y = x; return y;}` cause a dangling pointer when called? Why or why not?

    No, because `y` is an alias of `x`, not a pointer to `x`.

12. Draw a picture of the state of the world just before the following TypeScript script exits:

```
type Cell = { word: String, next?: Cell }
let c1: Cell = { word: "ready", next: { word: "set", next: {word: "go"}}}
let c2 = c1.next
let c3 = {...c2}
if (c2) c2.word = "ready"
```



13. Why is the stack “faster than the heap”?

    Allocation on the heap requires navigating some crazy data structure to find space, and deallocation requires the data structure to be updated to understand the changes. Stack allocation and deallocation is just a matter of updating a single processor register.

14. What is the difference between a string and a symbol?

    A string is a **decomposable sequence** of characters, while a symbol is an **atomic** value that can not be decomposed.

15. It’s 2024 and **Python still does not have a ?. operator**. Give a single Python expression that best approximates the JavaScript/Rust/Swift expression `winner?.team?.score`.

    ```
    winner.team.score if winner and winner.team else None
    ```

16. In Python, what is the list comprehension expression for all pairs (2-tuples) of integers where the first element is in the range 1..5 inclusive and the second is in the range 1..3 inclusive?

    ```
    [(i,j) for i in range(1,6) for j in range(1,4)]
    ```

17. Write a single expression to produce the product of each of the odd elements of the sequence named ***s*** in two of the following languages: JavaScript, Haskell, Java, Kotlin, Swift, or C++. Use

whatever version of `map` and `filter` and anonymous functions your language provides. Don't worry too much about syntax: you can still get full credit if you are very close.

```
# Haskell
product (filter odd s)

// Swift
s.filter{$0 % 2 != 0}.reduce(1, *)

// Kotlin
s.filter{it % 2 != 0}.reduce{a, b -> a * b}

// Java
s.stream().filter(x -> x % 2 != 0).reduce(1, (a, b) -> a * b)

// JavaScript
s.filter(x => x % 2 !== 0).reduce((a, b) => a * b, 1)

// C++
std::accumulate(s.begin(), s.end(), 1,
    [](int a, int b) {return a * (b % 2 != 0 ? b : 1);})
```

18. To Alan Kay, OOP is not so much about inheritance and polymorphism, but rather what?

    OOP is about **message passing**, **encapsulation**, and **late binding**.

19. Give a set of sealed interfaces or classes in either Java or Kotlin that serves as a translation of the following Swift enum. Don't panic! I'm not expecting syntactic perfection, but you should get pretty close.

```
enum Token {
    case plus
    case minus
    case identifier(String)
    case number(Float64)
}
```

```
// Java
sealed interface Token permits Plus, Minus, Identifier, Number {}
final class Plus implements Token {}
final class Minus implements Token {}
final class Identifier implements Token {String name;}
final class Number implements Token {double value;}

// Kotlin
sealed class Token {
```

```
    object Plus : Token()
    object Minus : Token()
    data class Identifier(val name: String) : Token()
    data class Number(val value: Double) : Token()
}
```

20. Programming language theory purists tend to make a huge point about types and classes being very different things. What are the main differences between types and classes, to these theoretical purists?

    To the purist, types are entirely **static** (used for type checking at compile time), while classes are **dynamic** (available to the program at run time for run-time type checking).

21. Give at least two examples of Java expressions whose static and dynamic type differ.

    After `List<Integer> x = new ArrayList<>();`, the static type of `x` is `List<Integer>` and its dynamic type is `ArrayList`, because Java **erases** the generic constraints.

    After `Object s = "dog";`, the static type of `s` is `Object` but its dynamic type is `String`.

22. In many programming languages, the option type is a true sum type; for example, in Haskell, its constructors are `Just` and `Nothing` and in Rust its constructors are `Some` and `None`. TypeScript doesn't have a built-in option type—but you can define it yourself! Show how to define the type `Option<T>` in TypeScript. (Hint: define two types for the present and missing options, then make the option type be the union of each.)

```
type Present<T> = {tag: "present", value: T}
type Missing = {tag: "missing"}
type Option<T> = Present<T> | Missing
```

23. This Rust code does not compile. What is wrong and how do you fix it?

```
struct Point {x: f64, y: f64}
fn mirror(p: Point) -> Point { Point{x: -p.x, y: -p.y} }
fn main() {
    let p = Point{x: 1.0, y: 2.0};
    let q = mirror(p);
    println!("({},{}) ({},{})", p.x, p.y, q.x, q.y);
}
```

    Here `p` is no longer the owner of the point object after calling `mirror`, so the expression `p.x` is a compile-time error! It can be fixed by making `mirror` borrow the point (e.g., `fn mirror(p: &Point)`).

Alternate answer: you can also say that the problem is that `Point` struct is not copyable, and you can make it so with an attribute definition.

24. What are two ways in Go to make the main goroutine wait for all other goroutines to finish? (Be careful not to do too much hand-waving in your answer. You don't *have* to write code, but you do need to be quite explicit about the details of the mechanisms.)

    One way is to use a **WaitGroup** `wg`. The main goroutine says `wg.Add(n)` where `n` is the number of other goroutines, then launches the goroutines, can calls `wg.Wait()`. Each of the goroutines calls `wg.Done()` when they finish.

    The other way is to use a **channel**. The main goroutine `make`s the channel, then launches the goroutines, then reads from the channel `n` times, where `n` is the number of other goroutines. Each goroutine writes to the channel when it finishes.

25. Write a code fragment in Go that tries to either (1) write the value 5 to the channel `c1`, or (2) read from the channel `c2` into the variable named `x`, but if it cannot do either within 5 seconds, prints the message `"oof"`. Assume that the `c1`, `c2`, and `x` are defined for you already.

```
select {
case c1 <- 5:
case x = <-c2:
case <-time.After(5 * time.Second):
    fmt.Println("oof")
}
```